Lycée Duhamel du Monceau. Niveau : Terminale.		
	Numérique et Sciences Informatiques	
Structures de données. Les classes. Correction.		

Jusqu'à présent nous avons utilisé des objets dont le type est prédéfini :
int , float, bool , str , list , tuple, dict.
Nous avons même appris à utiliser des méthodes associées à ces types, comme ma_liste.append(valeur).
Python, comme d'autres langages (Java, C++, ...), est un langage orienté objet. On peut même dire que tout y est objet.
Une variable de type int est en fait un objet de type int donc construit à partir de la classe int. Pareil pour les float et string. Mais également pour les list, tuple, dict, etc.
On peut créer ses propres objets qui auront leurs méthodes, ce qui aura pour effet :

- De faciliter l'implémentation de données.
- D'augmenter la lisibilité du programme.

Définition :

Une **classe** définit des objets qui sont des **instances** (des représentants) de cette classe. On utilisera le mot **objet** ou **instance** pour désigner la même chose. Les **objets** peuvent posséder des **attributs** (variables associées aux objets) et des **méthodes** (qui sont des fonctions associées aux objets et qui peuvent agir sur ces derniers ou encore les utiliser).

I) JE COMPRENDS PAR L'EXEMPLE :

Imaginons que nous ayons un objet Eleve, qui permettrait pour un élève de stocker dans une variable eleve1 des données comme son nom, son prénom, sa date de naissance et ses notes dans différentes matières, ainsi qu'une méthode qui calculerait sa moyenne.

1°) Création de la classe :

Pour créer une classe on utilise le mot-clé : class suivi par le nom de la classe et des incontournables " : ".
Le nom de la classe commence par une majuscule et ne contient que des caractères alphanumériques.

Méthode :

```
# création de la classe Eleve
class Eleve:
```

Ajoutons l'instruction pass et créons une variable eleve1 de type Eleve (*une instance de la classe Eleve*).

```
class Eleve:
   pass # signifie ne fait rien...

eleve1=Eleve()
print(type(eleve1))
```

Le résultat <class '__main__.Eleve'> signifie que eleve1 est une instance de la classe Eleve (une variable de type Eleve).

2°) Les attributs :

Les attributs sont des variables associées à la classe. Il y en a de deux types :

- Les attributs de classe : un attribut de classe (ou variable de classe) est un attribut qui sera identique pour chaque instance et n'a pas vocation à être changé.
- Les attributs d'instance : une variable ou attribut d'instance est une variable accrochée à une instance et qui est spécifique à cette instance. Et d'une instance à l'autre il ne prendra pas forcément la même valeur.

Pour notre exemple, nous choisirons les matières comme attributs de classe et les nom, prénom, date de naissance et les notes comme attributs d'instance.

Méthode :

Le constructeur :
L'endroit le plus approprié pour déclarer un attribut est à l'intérieur d'une méthode appelée le constructeur. S'il est défini, il est implicitement exécuté lors de la création de chaque instance. Le constructeur d'une classe se présente comme une méthode et suit la même syntaxe à ceci près que son nom est imposé : __init__ (deux underscores de chaque côté...). Hormis le premier paramètre, invariablement self, il n'existe pas de contrainte concernant la liste des paramètres excepté que le constructeur ne doit pas retourner de résultat.

```
class Eleve:
   # attributs de classe
   matiere1="Programmation"
   matiere2="Algorithmique"
   matiere3="Projet"
   # constructeur (attention deux underscores de chaque côté !!)
   def __init__(self,Nom,Prenom,Date,Note1,Note2,Note3):
      # attributs d'instance
      self.nom=Nom
      self.prenom=Prenom
      self.date=Date
      self.note_mat1=Note1
      self.note_mat2=Note2
      self.note_mat3=Note3
#création d'un élève
eleve1=Eleve("Térieur","Alain","01/01/2000",12,10,15)
```

Comment accéder aux valeurs des attributs d'instance créés ?
On utilise la syntaxe : instance.attribut (*notez bien le point).*

```
print(eleve1.prenom,eleve1.nom)
print(eleve1.matiere1,":",eleve1.note_mat1)
print(eleve1.matiere2,":",eleve1.note_mat2)
print(eleve1.matiere3,":",eleve1.note_mat3)
```

Question 1 :
Quel est l'affichage obtenu ?

```
On obtient :
Alain Térieur
Programmation : 12
Algorithmique : 10
Projet : 15
```

3°) Les Méthodes :
Il serait intéressant de pouvoir obtenir la moyenne de l'élève avec l'instruction eleve1.moyenne(). C'est à dire en appliquant la méthode moyenne() à l'instance eleve1. Pour cela on crée une fonction à l'intérieur de la classe Eleve qui retourne la moyenne de l'élève :

```
def moyenne(self):
   return ((self.note_mat1+self.note_mat2+self.note_mat3)/3)
```

À faire 1 :
Ajouter cette méthode et faire afficher la moyenne de l'élève.
Ajouter ces deux élèves et faire afficher leurs résultats et leurs moyennes.
• Nom : Onette
• Prénom : Camille
• Date : 01/07/2004
• Programmation : 7
• Algorithmique : 14
• Projet : 11

• Nom : Oma
• Prénom : Modeste
• Date : 01/11/2002

• Programmation : 13
• Algorithmique : 8
• Projet : 17

Correction :

```
classe1=[]

class Eleve:
    # attributs de classe
    matiere1="Programmation"
    matiere2="Algorithmique"
    matiere3="Projet"
    # constructeur (attention deux underscores de chaque côté !!)
    def __init__(self,Nom,Prenom,Date,Note1,Note2,Note3):
        # attributs d'instance
        self.nom=Nom
        self.prenom=Prenom
        self.date=Date
        self.note_mat1=Note1
        self.note_mat2=Note2
        self.note_mat3=Note3

    def moyenne(self):
        return ((self.note_mat1+self.note_mat2+self.note_mat3)/3)

#création des élèves
eleve1=Eleve("Térieur","Alain","01/01/2000",12,10,15)
eleve2=Eleve("Onette","Camille","01/07/2004",7,14,11)
eleve3=Eleve("Oma","Modeste","01/11/2002",13,8,17)

classe1.append(eleve1)
classe1.append(eleve2)
classe1.append(eleve3)

for eleve in classe1:
    print(eleve.prenom,eleve.nom)
    print(eleve.matiere1,":",eleve.note_mat1)
    print(eleve.matiere2,":",eleve.note_mat2)
    print(eleve.matiere3,":",eleve.note_mat3)
    print ('Moyenne :',eleve.moyenne())
```

Exercice 1 :

Écrire une fonction (hors de la classe bien sûr) qui prend en paramètre une liste constituée de ces trois élèves et qui retourne les moyennes par matières.

On attend le rendu :

```
{'Programmation': 10.666666666666666, 'Algorithmique': 10.666666666666666, 'Projet':
14.333333333333334}
```

Correction :

```
classe1=[]

class Eleve:
    # attributs de classe
    matiere1="Programmation"
    matiere2="Algorithmique"
    matiere3="Projet"
    # constructeur (attention deux underscores de chaque côté !!)
    def __init__(self,Nom,Prenom,Date,Note1,Note2,Note3):
        # attributs d'instance
        self.nom=Nom
        self.prenom=Prenom
```

```
        self.date=Date
        self.note_mat1=Note1
        self.note_mat2=Note2
        self.note_mat3=Note3

    def moyenne(self):
        return ((self.note_mat1+self.note_mat2+self.note_mat3)/3)

#création des élèves
eleve1=Eleve("Térieur","Alain","01/01/2000",12,10,15)
eleve2=Eleve("Onette","Camille","01/07/2004",7,14,11)
eleve3=Eleve("Oma","Modeste","01/11/2002",13,8,17)

classe1.append(eleve1)
classe1.append(eleve2)
classe1.append(eleve3)

for eleve in classe1:
    print(eleve.prenom,eleve.nom)
    print(eleve.matiere1,":",eleve.note_mat1)
    print(eleve.matiere2,":",eleve.note_mat2)
    print(eleve.matiere3,":",eleve.note_mat3)
    print ('Moyenne :',eleve.moyenne())

def moyennes_matieres(classe):
    som_mat1=0
    som_mat2=0
    som_mat3=0
    for eleve in classe:
        som_mat1=som_mat1+eleve.note_mat1
        som_mat2=som_mat2+eleve.note_mat1
        som_mat3=som_mat3+eleve.note_mat3
    moyennes={eleve.matiere1:som_mat1/len(classe),eleve.matiere2:som_mat2/len(classe),
    eleve.matiere3:som_mat3/len(classe)}
    return moyennes

print(moyennes_matieres(classe1))
```

II) LA DOCUMENTATION :

Notre classe Eleve est une structure de données qui peut être utilisée dans différents programmes par différents programmeurs, il est donc important voire primordial de bien la documenter.
Cette documentation doit montrer à minima comment sont construites les instances, quels sont les attributs et les méthodes disponibles.
Cette documentation est accessible via l'instruction : help(Eleve)
Par exemple voici une documentation possible :

```
class Eleve:
    '''
    Création d'une instance eleve:
    eleve=Eleve(nom(str),prenom(str),date(str),note1(programm)(float)
    note2(algo)(float),note3(projet(float)))
    attributs d'instance : nom,prenom,date,note_mat1,note_mat2,note_mat3
    attributs de classe: matiere1,matiere2,matiere3
    Méthode : moyenne() retourne la moyenne de l'élève
    '''
    .....
```

C'est d'autant plus important lorsque cette classe se trouve dans un autre fichier :

À faire 2 :
Copier-coller la classe Eleve dans un fichier que vous enregistrerez sous le nom Maclasse.py
Créer un autre fichier mon_programme.py, dans lequel vous écrirez :
(*les deux fichiers doivent être dans le même dossier*...)

```
from Maclasse import Eleve
eleve1=Eleve("Térieur","Alain","01/01/2000",12,10,15)
print(eleve1.prenom,eleve1.nom,":",eleve1.moyenne())
```

Si vous donnez cette classe à un autre programmeur, il faut qu'il puisse savoir comment l'utiliser sans avoir à en décortiquer le code...

Remarque :
Nous avons construit une structure de données répondant à un cahier des charges pas trop ambitieux. On pourrait se poser les questions suivantes :

- Que faire si un élève n'a pas de notes dans une des matières ?
- Comment rendre impossible la saisie d'une note supérieure à 20 ?
- Etc.

III) UN AUTRE EXEMPLE :

Python, comme la plupart des langages, ne dispose pas d'une structure de données de type rationnel (des fractions..)
Nous allons en créer une (même s'il est certain que cela existe déjà).

Le cahier des charges
On doit créer une classe Rationnel dont les instances auront les attributs numerateur et denominateur (celui-ci ne pourra être nul !!) et des méthodes pour :
• Simplifier la fraction et normaliser son écriture.
• Additionner deux fractions.
• Soustraire deux fractions.
• Multiplier et diviser deux fractions.

Méthode :
Un peu de Mathématiques.
La fraction 12/-15 doit s'écrire -4/5.
Il y a eu simplification par le PGCD de 12 et 15 et transfert du signe au numérateur. Il nous faudra :
• Une fonction qui calcule le PGCD.
• Une méthode pour simplifier et normaliser la fraction.

À faire 3 :
Créer un fichier ratio.py et écrire le code suivant :

```
# fichier ratio.py
class Rationnel:
    # création des instances
    def __init__(self,num,den=1):# par défaut le dénominateur vaut 1
        if den == 0:
            # on déclenche une exception spécifique
            raise ZeroDivisionError('denominateur nul')
        else:
            self.num=num
            self.den=den
    # pour voir une fraction sur la console appelée par print
    def __str__(self):
        return str(self.num)+'/'+str(self.den)
```

La méthode __str__ *permet de gérer l'affichage de la fonction* print.

Dans un autre fichier test.py, écrire le code suivant et expliquer les affichages dans la console.

```
from ratio import Rationnel
```

```
f=Rationnel(12,-15)
print(f)
print(f.num,f.den)

f1=Rationnel(7)
print(f1)
print(type(f1))
```

On obtient :

```
12/-15                    # affichage de f
12 -15                    # f.num : 7 et f.den : -15
7/1                       # c'est 7 sous forme de rationnel
<class 'ratio.Rationnel'> # print(type(f1))
```

Simplification de la fraction

Pour cela il nous faut une fonction qui calcule le PGCD de deux entiers.

À faire 4 :

Un peu d'algorithmique.

Écrire une fonction pgcd(x,y) qui retourne le pgcd des entiers x et y passés en paramètres et la mettre dans le fichier ratio.py (en dehors de la classe). **Aide** : chercher Algorithme d'Euclide.

```
def pgcd(x,y):
    # code
    return ...

class Rationnel:
    # création des instances
    def __init__(self,num,den=1):# par défaut le dénominateur vaut 1
...
```

Correction :

En mathématiques, l'**algorithme d'Euclide** est un algorithme qui calcule le plus grand commun diviseur (PGCD) de deux entiers, c'est-à-dire le plus grand entier qui divise les deux entiers, en laissant un reste nul. L'algorithme ne connaît pas la factorisation de ces deux nombres.

Le PGCD de deux entiers relatifs est égal au PGCD de leurs valeurs absolues : de ce fait, on se restreint dans cette section aux entiers positifs. L'algorithme part du constat suivant : le PGCD de deux nombres n'est pas changé si on remplace le plus grand d'entre eux par leur différence. Autrement dit, pgcd(a, b) = pgcd(b, a - b). Par exemple, le PGCD de 252 et 105 vaut 21, mais c'est aussi le PGCD de 252 - 105 = 147 et 105. Ainsi, comme le remplacement de ces nombres diminue strictement le plus grand d'entre eux, on peut continuer le processus, jusqu'à obtenir deux nombres égaux.

Dans l'algorithme, on pourrait réaliser autant de différences que l'on souhaite. Par exemple, le PGCD de 252 et 105 est aussi égal au PGCD de 105 et 252 - 2 × 105 = 42. Ainsi, l'algorithme d'Euclide opère ainsi : on remplace le plus grand des deux nombres par le reste de la division euclidienne du plus grand nombre par le plus petit. Par exemple, la division euclidienne de 252 par 105 donne un reste de 42. Dans Introduction à l'algorithmique de Cormen et al. (voir Th. 31.9[13]), les auteurs appellent *théorème de la récursivité pour le PGCD* le fait suivant :

pgcd(a, b) = pgcd(b, r) où r est le reste de la division euclidienne de a par b.

Ainsi, l'algorithme d'Euclide sur deux nombres entiers positifs a et b avec $a > b \geq 0$ procède comme suit :

- Si $b = 0$, l'algorithme termine et rend la valeur a ;
- Sinon, l'algorithme calcule le reste r de la division euclidienne de a par b, puis recommence avec $a = b$ et $b = r$.

Formellement l'algorithme d'Euclide construit deux suites finies d'entiers par récurrence : la suite (q_n) des quotients et la suite (r_n) des restes :

- $r_0 = a$, $r_1 = b$;
- pour $n \geq 1$, r_{n+1} et q_n sont le reste et le quotient de la division euclidienne de r_{n-1} par r_n, définis par les deux conditions :
    - $r_{n-1} = r_n q_n + r_{n+1}$ et
    - $0 \leq r_{n+1} < r_n$.

Par définition la suite (r_n) est une suite strictement décroissante d'entiers positifs : elle est donc finie et s'arrête au premier n tel que $r_n = 0$.

Exemple

Le tableau suivant montre le calcul du PGCD de 21 et 15. On réalise la division euclidienne de $a = 21$ et $b = 15$: le quotient est 1 et le reste 6. L'algorithme continue avec $a = b$ et b = le précédent reste, jusqu'à trouver un reste nul. L'algorithme s'arrête alors et retourne le dernier reste non nul trouvé, ici $r_3 = 3$ qui est bien le PGCD de 21 et 15 :

Étape *n*	**Dividende**	**Diviseur**	**Équation**	**Quotient**	**Reste**
1	21	15	$21 = 15 \times 1 + 6$	1	6
2	15	6	$15 = 6 \times 2 + 3$	2	3
3	6	3	$6 = 3 \times 2 + 0$	2	0
4	3	0	fin de l'algorithme		

Source : Wikipédia.

Codage du PGCD en version récursive :

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

def pgcd(a,b):
    """pgcd(a,b): calcul du 'Plus Grand Commun Diviseur' entre les 2 nombres entiers a et b"""
    if b==0:
        return a
    else:
        r=a%b
        return pgcd(b,r)
```

```
# Exemple d'utilisation:
print pgcd(56,42) # =>  affiche 14
```

Codage du PGCD dans sa version non-récursive (à cause des limites de la pile Python) :

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

def pgcd(a,b):
    """pgcd(a,b): calcul du 'Plus Grand Commun Diviseur' entre les 2 nombres entiers a et b"""
    while b!=0:
        r=a%b
        a,b=b,r
    return a
```

```
# Exemple d'utilisation:
print pgcd(56,42) # =>  affiche 14
```

<u>À faire 5</u> :

Voici la fonction à rajouter dans la classe Rationnel :

```
#simplification des fractions
def normalise(self):
    g = pgcd(abs(self.num), abs(self.den))
    self.num = self.num // g
    self.den = self.den //g
    if(self.num*self.den<0):
        if self.den<0:
            self.den=-self.den
            self.num=-self.num
    else:
```

```
        if self.den <0:
            self.den=-self.den
            self.num=-self.num
```

Écrire un algorithme en langage naturel qui représente cette fonction :

Correction :

```
fonction normalise :
        g ← pgcd(num,den)
        num←num//g    # // quotient division euclidienne
        den←den//g
        si num*den<0 alors :
                si den<0 alors :
                        den←-den
                        num←-num
        sinon :
                si den<0 alors :
                        den←-den
                        num←-num
```

Pour que la fraction soit automatiquement simplifiée et normalisée, il suffit de l'exécuter dans la fonction __init__:

```
def __init__(self,num,den=1):
    if den == 0:
        # on déclenche une exeption spécifique
        raise ZeroDivisionError('denominateur nul')
    else:
        self.num=num
        self.den=den
        self.normalise()
```

À faire 6 :

Exécuter de nouveau le code dans le fichier test.py et vérifier que l'on obtient bien :

```
-4/5
-4 5
7/1
<class 'ratio.Rationnel'>
```

On obtient bien :

```
-4/5
-4 5
7/1
<class 'ratio.Rationnel'>
```

Une première méthode : l'addition

On pourrait imaginer une fonction add(self, autre) :

```
def add(self,other):
    return Rationnel(self.num*other.den+self.den*other.num,self.den*other.den)
```

Mais l'écriture d'une addition de deux fractions f1+f2 serait f1.add(f2).

```
f1=Rationnel(2,7)
f2=Rationnel(5,3)
print(f1.add(f2))
affiche 41/21
```

Pensons alors à l'utilisateur qui s'attend à pouvoir écrire print(f1+f2)

Heureusement nous pouvons définir l'addition '+' pour nos rationnels...
Il existe une fonction prédéfinie pour cela : __add__:

```
def __add__(self, other): #addition
    n=self.num*other.den+other.num*self.den
    d=self.den*other.den
    return Rationnel(n,d)
```

À faire 7 :
Rajouter cette méthode dans la classe Rationnel et testez le code suivant dans test.py

```
f1=Rationnel(2,7)
f2=Rationnel(5,3)
print(f1+f2)
```

Les autres opérations
• Pour la soustraction '-' on utilise la fonction : __sub__(self,other).
• Pour la multiplication '*' on utilise la fonction : __mul__(self,other).
• Pour la division "/" ce sera : __truediv__(self,other), il faudra gérer la division par 0.

À faire 8 :
Réaliser ces méthodes et les tester.
Correction :
On ajoute les fonctions correspondantes dans la classe Rationnel :

```
def __sub__(self,other) : #soustraction
    n=self.num*other.den-other.num*self.den
    d=self.den*other.den
    return Rationnel(n,d)
```

```
def __mul__(self,other) : #multiplication
    n=self.num*other.num
    d=self.den*other.den
    return Rationnel(n,d)
```

```
def __truediv__(self,other) : #division
        if (other.den or self.den) == 0:
            # on déclenche une exception spécifique
            raise ZeroDivisionError('denominateur nul')
        else:
            n=self.num/other.num
            d=self.den/other.den
            return Rationnel(n,d)
```

Exercice 2 :
Le nombre d'Euler : e = 2,718281.... vérifie l'égalité suivante :
$e = 1/1 + 1/(1\times2) + 1/(1\times2\times3) + ::: + 1/(1\times2\times3\times:::\times n) + ...$
En utilisant la classe Rationnel, écrire une fonction nombre_euler(n) qui retourne une fraction qui permet d'obtenir une approximation de e.
Correction :

```
def __euler__(self,n):
    '''approximation de e avec n etapes de la methode d'Euler'''
    e = 1
    deno = 1
    for k in range(1,n+1):
        deno = deno * k
        e = e + 1/(deno)
    return Rationnel(e)
```

Exercice 3 :

Toujours en utilisant la classe Rationnel, écrire une fonction qui permet d'obtenir une fraction donnant une approximation du nombre pi par la formule :
$\pi = 4 \times (1- 1/3 + 1/5 – 1/7 + 1/9 + ….. + (-1)^n/(2n+1)+…)$

Correction :
```
def __pi__(self,n):
    '''approximation de pi avec n etapes'''
    somme = 0
    for k in range(0,n+1):
        somme = somme + (((-1)**k)/(2*k+1))
    pi = 4 * somme
    return Rationnel(pi)
```

Prolongement possible
Pour un projet, on peut imaginer de développer davantage la structure Rationnel en y ajoutant d'autres méthodes qui permettent de comparer des rationnels, de calculer $(a/b)^n$ etc.
En documentant l'ensemble bien sûr...

IV) EXERCICES :

Exercice 4 :
Écrire une classe Domino pour représenter une pièce de domino. Les objets sont initialisés avec les valeurs des deux faces, A et B.
Ajouter une méthode affichePoints(self) qui affiche les valeurs des deux faces, et une méthode total(self) qui retourne la somme des deux valeurs.
Correction :
```
class Domino :
        # Constructeur
        def __init__(self, A=0, B=0):
                self.A = A
                self.B = B

        # Méthode
        def affiche_point(self):
                print ("face A : {} face B : {}".format(self.A,self.B))

        def valeur(self):
                somme = self.A + self.B
                return somme

#Programme Principal

d1 = Domino(2,6)
d2 = Domino(4,3)
```

Exercice 5 :
Écrire une classe CompteBancaire. Les objets sont initialisés avec le nom du titulaire et le solde. L'argument solde doit être facultatif et avoir une valeur prédéfinie zéro.
Ajouter deux méthodes depot(self,somme) et retrait(self,somme) pour changer le solde. Ajouter une méthode affiche(self) qui montre le solde courant.
Correction :
```
class CompteBancaire:
    def __init__(self, nom='Dupont', solde=1000):
        self.nom=nom
        self.solde=solde

    def depot(self,a):
        self.solde = self.solde + a

    def retrait(self,a):
```

```
        self.solde = self.solde - a

    def affiche(self):
        texte='Le solde du compte bancaire de '+str(self.nom)+' est de '+str(self.solde)+' euros'
        print texte

compte1=CompteBancaire('Duchmol', 800)
compte1.depot(350)
compte1.retrait(200)
compte1.affiche()
```

Exercice 6 :
Écrire une classe Rectangle, permettant de construire un rectangle doté d'attributs longueur et largeur.
Ajouter deux méthodes perimetre(self) et surface(self).
Correction :

```
class Rectangle(object):
    "Classe de rectangles"
    def __init__(self, longueur =0, largeur =0):
        self.L = longueur
        self.l = largeur
        self.nom ="rectangle"

    def perimetre(self):
        return "(%d + %d) * 2 = %d" % (self.L, self.l,(self.L + self.l)*2)

    def surface(self):
        return "%d * %d = %d" % (self.L, self.l, self.L*self.l)

if __name__ == "__main__":
    r1 = Rectangle(15, 30)
    print('Périmètre :',r1.perimetre())
    print('Surface :',r1.surface())
```

Exercice 7 :
Écrire une classe Personnage, avec son nom et ses points de vie comme attribut.
Ajouter les méthodes donneEtat(self) qui indique si le personnage est vivant, une méthode perdVie qui fait diminuer de manière aléatoire le nombre de point de vie d'un personnage, et une méthode combat(self,autre) qui permet d'affronter deux personnages et indique quel est celui qui a gagné avec son nombre de points de vie restants.
Correction :

```
import random
class Personnage:
    def __init__(self, Nom, nbreDeVie):
        self.nom=Nom
        self.vie=nbreDeVie
    def donneEtat (self):
        return self.vie
    def perdVie (self):
        if random.random()>0.5:
            nbPoint = 1
        else :
            nbPoint = 2
        self.vie=self.vie-nbPoint

    def combat(self,autre):
        while self.donneEtat()>0 and autre.donneEtat()>0 :
            self.perdVie()
            autre.perdVie()
        if self.donneEtat()<=0 and autre.donneEtat()>0:
```

```
        msg = str(autre.nom)+" est vainqueur, il lui reste encore "+str(autre.donneEtat())+" points alors que
"+str(self.nom)+" est mort"
    elif autre.donneEtat()<=0 and self.donneEtat()>0:
        msg = str(self.nom)+" est vainqueur, il lui reste encore "+str(self.donneEtat())+" points alors que
"+str(autre.nom)+" est mort"
    else :
        msg = "Les deux combattants sont morts en même temps"
    return msg

bilbo = Personnage('Bilbo',20)
gollum = Personnage('Gollum',20)
print(bilbo.combat(gollum))
```

Exercice 8 :
Écrire une classe Robot, avec ses coordonnées comme attributs et une direction (None par défaut).
Ajouter la méthode avancer(self), qui permet au robot d'avancer d'une case dans la direction choisie et la méthode pivoter(self) qui lui permet de pivoter de 90° vers la droite.
Correction :

```
class Robot:
    """ Robot qui sait avancer d'une case et pivoter à droite de 90°.
    Il est repéré par son abscisse x, son ordonnée y et sa direction.
    """
    def __init__(self, x, y, direction=None):
        """ Initialiser le robot self à partir de sa position (x, y) et sa direction. """
        self.x = x
        self.y = y
        self.direction = 'nord est sud ouest'.split().index(direction)
    def avancer(self):
        """ Avancer d'une case dans la direction"""
        # mettre à jour self.x
        if self.direction == 1: # est
            self.x += 1
        elif self.direction == 3: # ouest
            self.x -= 1
        # mettre à jour self.y
        if self.direction == 0: # nord
            self.y += 1
        elif self.direction == 2: # sud
            self.y -= 1

    def pivoter(self):
        """ Pivoter ce robot de 90° vers la droite. """
        self.direction = (self.direction + 1) % 4
    def afficher(self, *, prefix=''):
        print(prefix + '({}, {})>{}'.format(self.x, self.y, 'nord est sud ouest'.split()[self.direction]))

# Exemple
r1 = Robot(4, 10, 'est')
r1.afficher(prefix='r1 = ')
r2 = Robot(15, 7, 'sud')
r2.afficher(prefix='r2 = ')
r1.pivoter()
r1.afficher(prefix='r1 = ')
r2.avancer()
r2.afficher(prefix='r2 = ')
```